

Predicting Reaction Enthalpy via Elemental Magpie and Structural SOAP Fingerprints

Building two ML pipelines :

Approach 1 : Using Matminer/Magpie elemental descriptors

Approach 2 : Using DFT crystal structures from the Materials Project with SOAP fingerprints

to predict the standard enthalpy of chemical reactions.

Amish Verma (230121)

Tanvi Manhas (231084)

Predicting ΔH from Reaction Data

Data of reactions:

We extract JSON dataset of chemical reactions, each containing reactants and products with stoichiometric coefficients, equilibrium temperature T_{eq} (K), and reaction enthalpy ΔH in kJ/mol (the prediction target)

The benefit of ML:

- DFT computation of ΔH is expensive, a trained ML surrogate model enables fast screening
- Reaction enthalpy is critical in thermodynamic databases, materials discovery, and process design

Data Observations and Cleaning :

- Outliers present in both T_{eq} and ΔH , detected using boxplots (Matminer) and IQR filtering (SOAP)
- IQR bounds: lower = $Q1 - 1.5 \times IQR$, upper = $Q3 + 1.5 \times IQR$

Approach 1 — Matminer/Magpie Elemental Descriptor

01

Step 1 — Load and parse reactions:

All unique compound formulas are extracted from reactants and products across every reaction in the JSON file.

02

Step 2 — Featurize compounds using Magpie preset:

Each formula is converted to a pymatgen Composition object. The ElementProperty featurizer with the "magpie" preset computes statistical summaries (mean, std, min, max, range) over all constituent elements' properties. This produces 132 features per compound, including atomic number, electronegativity, atomic radius, melting point, and valence electron count.

03

Step 3 — Build one reaction vector per reaction:

For each reaction, the feature vector is constructed by summing stoichiometric coefficient × compound feature vector for all reactants, then doing the same for all products.

04

Step 4 — Scale features and targets:

StandardScaler is applied independently to X (features), Teq, and ΔH .

Approach 1 — Matminer/Magpie Elemental Descriptor Pipeline

Code snippet:

```
ep = ElementProperty.from_preset("magpie")
features_df = ep.featurize_dataframe(compounds_df, col_id="composition")
rxn_vec = np.zeros(features_only.shape[1])
for comp, coeff in r["reactants"].items():
    rxn_vec += coeff * features_only.loc[comp].values
for comp, coeff in r["products"].items():
    rxn_vec += coeff * features_only.loc[comp].values
```

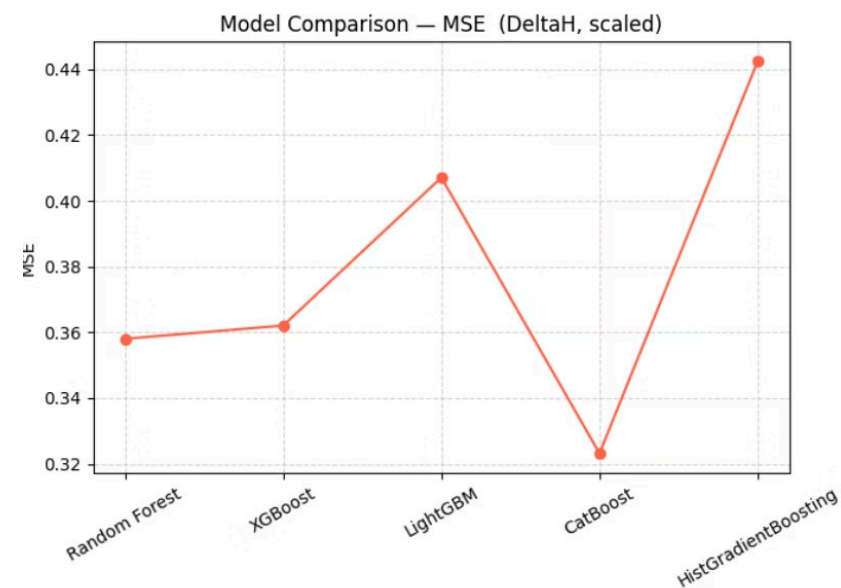
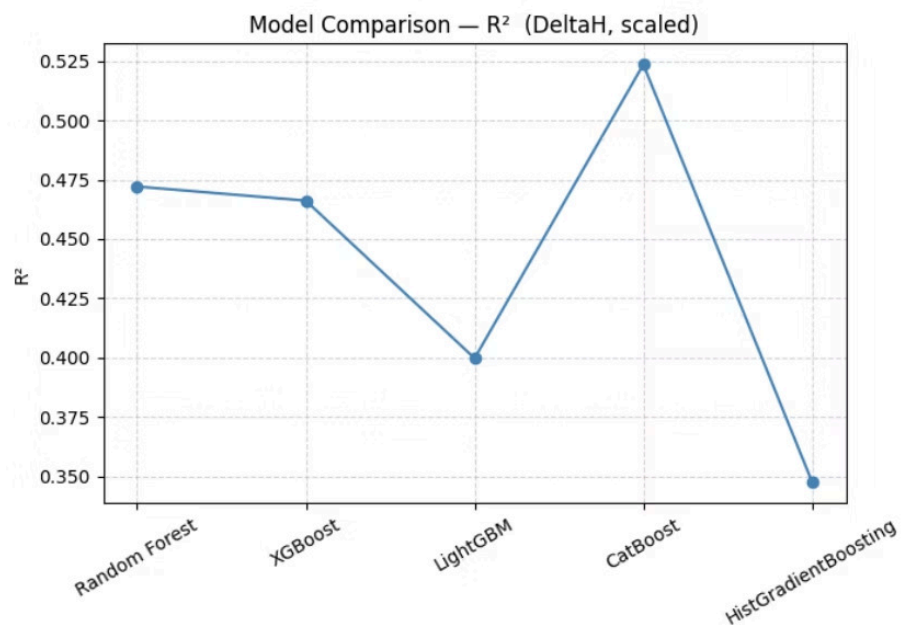
Google Colab Notebook : <https://colab.research.google.com/drive/1GUfGsEXg1ritYzkz5Cix-uH45SseCTxs?usp=sharing>

Approach 1 — Model Benchmarking and Results

Train/Test split: 80% train, 20% test, random_state=42

Models Trained on ΔH

Model	Hyperparameters
Random Forest	400 trees
XGBoost	500 estimators, learning_rate=0.05, max_depth=6, subsample=0.8
LightGBM	500 estimators, learning_rate=0.05, num_leaves=31, subsample=0.8
CatBoost	500 iterations, learning_rate=0.05, depth=6, RMSE loss
HistGradientBoosting	500 iterations, learning_rate=0.05, min_samples_leaf=20



Key Findings:

Magpie features provide a reasonable baseline but lack any 3D structural information, which limits accuracy on chemically diverse reaction sets.

The best result obtained overall was an $R^2 = 0.54$ with the tuned **CatBoost** model.

Approach 2 — SOAP Fingerprinting with DScribe

Part A — Outlier removal using IQR:

ΔH values are filtered using the interquartile range method. Reactions with enthalpy outside $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ are removed before any featurization. This is more principled than the visual boxplot inspection used in Approach 1.

Code snippet:

```
Q1 = np.percentile(enthalpies, 25)
Q3 = np.percentile(enthalpies, 75)
IQR = Q3 - Q1
filtered_data = [rxn for rxn in data if Q1 - 1.5*IQR <= rxn["delta_H_kj_mol"] <= Q3 + 1.5*IQR]
```

Google Colab Notebook : https://colab.research.google.com/drive/1PG75gZm4NI-TJ4_sdbcK-NY-5beU8dn7?usp=sharing

Part B — Materials Project API pipeline:

01

All unique compound formulas extracted from filtered reactions

02

MPRester queries each formula to retrieve its material_id — the most stable polymorph (lowest formation energy)

03

Compounds where material_id is "None" are tracked and flagged

04

Crystal structures downloaded via get_structure_by_material_id() and saved as CIF files

Materials Project provides real DFT-optimized periodic crystal structures — encoding far richer chemistry than elemental property averages.

Approach 2 — SOAP Fingerprinting with DDescribe

Smooth Overlap of Atomic Positions encodes the local atomic environment around each atom as a descriptor that is invariant to rotation, translation, and atomic permutation. It captures bond geometry, coordination environment, and 3D structure.

- **r_cut = 5.0 Å** (local environment cutoff radius)
- **n_max = 8** (radial basis functions)
- **l_max = 6** (angular resolution)
- **compression mode = mu2**
- **periodic = True** for crystal structures
- Result averaged across all atoms → one fixed-length vector per compound
- Saved as .npy files

Code snippet:

```
soap = SOAP(  
    species=list(set(atoms.get_chemical_symbols())),  
    compression={'mode': 'mu2'},  
    periodic=True,  
    r_cut=5.0,  
    n_max=8,  
    l_max=6  
)  
soap_vec = soap.create(atoms).mean(axis=0)  
np.save(f"soap_vec_{formula}.npy", soap_vec)
```

Handling molecular and elemental species (O₂, CO₂, H₂O, Hg, Ga, Cs):

These do not have crystal structures. The ASE molecule() builder is used to construct their geometry, and SOAP is computed with periodic=False.

Approach 2 — Exception Handling

Some compounds have fractional oxygen indices — for example WO_{2.96}, WO_{2.72}, WO_{2.9}, TbO_{1.72}, TbO_{1.81} — which do not exist as discrete entries in the Materials Project and therefore have no CIF structure. A non-stoichiometric oxide is thermodynamically intermediate between its bounding phases, so a weighted SOAP interpolation is a physically grounded approximation.

Weighted linear interpolation between nearest phases:

- $\text{WO}_{2.96} = 0.96 \times \text{WO}_3 + 0.04 \times \text{WO}_2$
- $\text{WO}_{2.72} = 0.72 \times \text{WO}_3 + 0.28 \times \text{WO}_2$
- $\text{WO}_{2.9} = 0.90 \times \text{WO}_3 + 0.10 \times \text{WO}_2$
- $\text{TbO}_{1.72} = 0.4399 \times \text{TbO}_2 + 0.28 \times \text{Tb}_2\text{O}_3$
- $\text{TbO}_{1.81} = 0.6199 \times \text{TbO}_2 + 0.19 \times \text{Tb}_2\text{O}_3$

```
for base_formula, weight in item["components"]:  
    base_vec = np.load(f"soap_vec_{base_formula}.npz")  
    result_vector += weight * base_vec  
np.save(f"soap_vec_{target_formula}.npz", result_vector)
```

Approach 2 — Reaction Vector Construction and CatBoost Results

Reaction vector construction:

For each reaction, the signed SOAP sum is computed — reactant vectors are subtracted and product vectors are added, consistent with the thermodynamic definition $\Delta H = H_{\text{products}} - H_{\text{reactants}}$.

```
vec = np.zeros_like(soap_vec)
for r in rxn["reactants"]:
    vec -= r["coeff"] * np.load(f"soap_vec_{r['formula']}.npy")
for p in rxn["products"]:
    vec += p["coeff"] * np.load(f"soap_vec_{p['formula']}.npy")
```

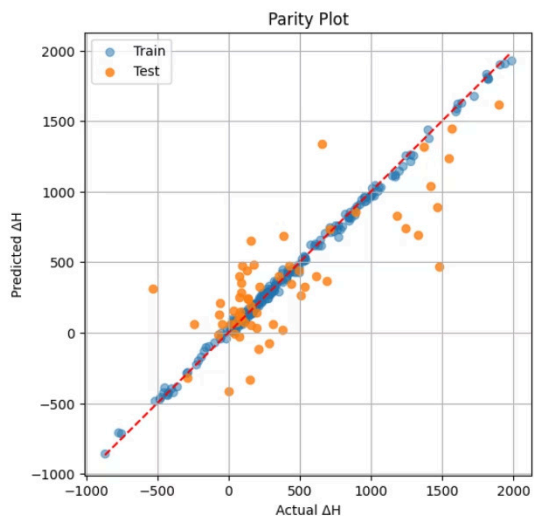
Only reactions where all compounds have a saved SOAP vector are included. Skipped reactions are tracked and reported.

Final dataset cleaning:

A final pass filters out any reactions with missing SOAP files before building the X matrix and y target array.

Model: CatBoost Regressor

- iterations=500, learning_rate=0.05, depth=6
- Train/test split: 80/20, random_state=42



MAE: 239.66971232938863

R^2 : 0.6457696874095108

Summary, Comparison, and Future Directions

Metric	Approach 1 (Matminer)	Approach 2 (SOAP)
Feature type	Elemental statistics (Magpie)	3D structural fingerprints (SOAP)
Feature source	Matminer presets	Materials Project API + DScribe
Data cleaning	Visual boxplot inspection	IQR-based outlier removal
Fractional compounds	Not necessary	Linearly interpolated between phases
Molecular species	Not applicable	Handled via ASE molecule builder
Models compared	RF, XGBoost, LightGBM, CatBoost, HGB	RF, CatBoost
Best R ²	0.54, CatBoost	0.65, CatBoost

Why SOAP performs better:

SOAP captures bond geometry, coordination environment, and crystal periodicity, which is information that Magpie elemental averages completely discard. The signed reaction vector also correctly encodes the directionality of ΔH .

Limitations:

- Materials Project structures are ground-state phases; reactions may involve metastable or amorphous intermediates
- SOAP vector size is species-dependent; reactions with unseen elements cannot be featurized without recomputing descriptors

Future directions:

- Apply graph neural networks (MEGNet, CGCNN) directly on crystal graphs for end-to-end learning
- Include Teq as an input feature for joint or multi-target modeling
- Expand the reaction dataset to cover more compound families and temperature ranges

Thank You!